

P3315R0

2024-07 Library Evolution Poll Outcomes

Published Proposal, 2024-08-15

Authors:

[Inbal Levi - Library Evolution Chair](#) (MPGC Services LTD)

[Fabio Fracassi - Library Evolution Assistant Chair](#) (CODE University of Applied Sciences)

[Ben Craig - Library Evolution Assistant Chair](#) (Raven)

[Billy Baker - Library Evolution Incubator Chair](#) (NVIDIA)

[Nevin Liber - Library Evolution Incubator Assistant Chair and Admin Chair](#) (Argonne National Laboratory)

[Corentin Jabot - Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

WG21

Table of Contents

1 Introduction

2 Poll Outcomes

3 Selected Poll Comments

3.1 Poll 1: Send "[P2835R4] Expose std::atomic_ref's object address" to Library Working Group for C++26.

3.2 Poll 2: Send "[P3016R3] Resolve inconsistencies in begin/end for valarray and braced initializer lists" to Library Working Group for C++26.

References

Informative References

§ 1. Introduction

In 2024-07, the C++ Library Evolution group conducted a series of electronic decision polls [\[P3314R0\]](#). This paper provides the results of those polls and summarizes the results.

In total, 19 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the papers' authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Send " [P2835R4] Expose std::atomic_ref's object address" to Library Working Group for C++26.	6	7	3	1	1	Consensus in favor
Poll 2: Send " [P3016R3] Resolve inconsistencies in begin/end for valarray and braced initializer lists" to Library Working Group for C++26.	6	6	0	1	1	Consensus in favor

All the polls have consensus in favor and will be forwarded to LWG for C++26.

§ 3. Selected Poll Comments

For some of the comments, small parts were removed to anonymize.

§ 3.1. Poll 1: Send "[P2835R4] Expose std::atomic_ref's object address" to Library Working Group for C++26.

This extension to std::atomic_ref's API vastly increases its usefulness in the scenarios described in its paper as justification.

— Strongly Favor

Returning a uintptr_t is probably the right balance for not making it too easy to use the object address incorrectly.

— Strongly Favor

no reason to make this optional. uintptr_t should be available unconditionally on all platform

— Weakly Favor

While this is a relatively minor use case, it is only sensible that a facility that constitutes a special reference to an object be able to distinguish that object from others of the same type. The resistance to error offered by the use of std::uintptr_t is valuable, although adding the generally useful std::copy_cv_t might have made using std::copy_cv_t * more palatable in terms of complexity. Presumably the editors can decide to use the correct feature-test macro value.

— Weakly Favor

Seems well motivated. I suggest avoiding the C-style cast in the specification (when it comes to wording).

— Weakly Favor

The performance optimizations for apps that want to move from the incorrect volatile T* to a correct atomic_ref are important and help fulfill the design intent of atomic_ref.

— Weakly Favor

Not completely convinced by the motivation

— Neutral

`uintptr_t` makes this not usable in `constexpr`, `T *` would be preferable

— Strongly Against

§ 3.2. Poll 2: Send "[P3016R3] Resolve inconsistencies in `begin/end` for `valarray` and braced initializer lists" to Library Working Group for C++26.

The changes to `initializer_list` are a definite improvement.

— Strongly Favor

All for resolving inconsistencies...!

— Strongly Favor

C++ experts criticize `valarray`, but it does have users. (Perhaps they keep quiet to avoid attracting attention!) Those users probably would appreciate some fixes.

The author's ad-hoc approach to `valarray` vs. `_braced_initializer_list` is reasonable. I think it's reasonable to add `data` and `empty` to `initializer_list`. It's perhaps a wrinkle in the Standard that `initializer_list` and `span` overlap even more than they did before. The proposed breaking changes (in use of `std::data` and `std::empty`, and in "availability of `begin / end`") are improvements to the language.

The author's explanation why the offending `std::begin` and `std::end` declarations in `<initializer_list>` can't be deprecated is reasonable.

I'm voting WF and not SF because I don't have a full grasp of how this proposal affects [LWG3624] and [LWG3625].

— Weakly Favor

The changes to `std::initializer_list` and to `std::valarray` should be distinct proposals. They might combine to come up in related situations, but the changes are quite distinct.

— Weakly Against

does not solve a real problem

— Strongly Against

§ References

§ Informative References

[P2835R4]

Gonzalo Brito Gadeschi. *Expose std::atomic_ref's object address*. 21 May 2024. URL: <https://wg21.link/p2835r4>

[P3016R3]

Arthur O'Dwyer. *Resolve inconsistencies in begin/end for valarray and braced initializer lists*. 22 March 2024. URL: <https://wg21.link/p3016r3>

[P3314R0]

Inbal Levi, Fabio Fracassi, Ben Craig, Nevin Liber, Billy Baker, Corentin Jabot. *2024-07 Library Evolution Polls*. 16 July 2024. URL: <https://wg21.link/p3314r0>